

SECTION 2 – Report / Analysis Queries (Screenshots)

pending_payments.png

This query retrieves a list of members who have pending payments. It joins the `Members` and `Payments` tables using `MemberID` and filters for `Status = 'Pending'`. The output displays each member's full name, the payment date, the amount due, and the current payment status.

Useful for identifying outstanding debts and prompting follow-ups.

class_schedule.png

This query displays the full class schedule, organised by trainer. It joins the `Trainers`, `ClassSchedule`, and `Classes` tables to show each trainer's name, the class they are teaching, and the scheduled date and time. The results are ordered by trainer last name and date.

Ideal for internal scheduling and publishing timetables.

most_booked_classes.png

This query identifies the top 5 most booked classes. It joins the `Bookings`, `ClassSchedule`, and `Classes` tables, counts the number of bookings using `COUNT(BookingID)`, and groups results by class name. The final output is ordered by popularity (`DESC`) and limited to 5 rows using `LIMIT 5`.

Helps the gym prioritise popular offerings and allocate resources effectively.

revenue_per_membership.png

This query calculates the total revenue generated per membership type. It joins the `Payments`, `Members`, and `Memberships` tables, filters for only successful payments (`Status = 'Paid'`), and sums the payment amounts grouped by membership type.

Gives insight into which memberships are the most financially beneficial.

member_attendance.png

To enhance the usefulness of the member attendance report, I refined this query to display each member's first and last name, the name of the class they attended, and a human-readable attendance status. I combined the class date and time into a single `ScheduledDateTime` field using `CONCAT(cs.Date, ' ', cs.StartTime)`, and translated binary attendance values (1 or 0) into 'Attended' or 'Missed' using a `CASE` statement.

These improvements make the report far more accessible and meaningful for staff monitoring member engagement.

most_booked_trainer.png

This query finds the trainer with the highest number of class bookings. It joins the `Trainers`, `ClassSchedule`, and `Bookings` tables, counts the bookings per trainer using `COUNT`, groups them, and orders the result from highest to lowest. The result is restricted to the top performer using `LIMIT 1`.

Helps reward high-performing trainers and optimise scheduling.

The screenshot shows a database query editor interface. At the top, there's a toolbar with various icons and a dropdown menu set to "Limit to 1000 rows". Below the toolbar, the SQL query is displayed in a text area:

```
1 • USE gymdb;
2 • SELECT t.FirstName AS TrainerFirstName, t.LastName AS TrainerLastName, c.Name AS ClassName,
3       cs.Date, cs.StartTime
4 FROM Trainers t
5 JOIN ClassSchedule cs ON t.TrainerID = cs.TrainerID
6 JOIN Classes c ON cs.ClassID = c.ClassID
7 ORDER BY t.LastName, cs.Date;
8
```

Below the query editor, there's a section labeled "Action Output" which displays a table of execution results:

	Time	Action	Response	Duration / Fetch Time
1	17:53:39	CREATE DATABASE gymdb	1 row(s) affected	0.012 sec
2	17:53:39	USE gymdb	0 row(s) affected	0.00058 sec
3	17:57:48	USE gymdb	0 row(s) affected	0.018 sec

```
1 • USE gymdb;
2 • SELECT c.Name AS ClassName, COUNT(b.BookingID) AS TotalBookings
3 FROM Bookings b
4 JOIN ClassSchedule cs ON b.ScheduleID = cs.ScheduleID
5 JOIN Classes c ON cs.ClassID = c.ClassID
6 GROUP BY c.Name
7 ORDER BY TotalBookings DESC
8 LIMIT 5;
```

```
1 • USE gymdb;
2 • SELECT ms.Type AS MembershipType, SUM(p.Amount) AS TotalRevenue
3 FROM Payments p
4 JOIN Members m ON p.MemberID = m.MemberID
5 JOIN Memberships ms ON m.MembershipID = ms.MembershipID
6 WHERE p.Status = 'Paid'
7 GROUP BY ms.Type;
```

```
1 • USE gymdb;
2 • SELECT m.FirstName, m.LastName, c.Name AS ClassName, cs.Date, b.Attendance
3 FROM Bookings b
4 JOIN Members m ON b.MemberID = m.MemberID
5 JOIN ClassSchedule cs ON b.ScheduleID = cs.ScheduleID
6 JOIN Classes c ON cs.ClassID = c.ClassID
7 WHERE m.MemberID = 1;
```

100% 22:7

Action Output

	Time	Action	Response	Duration / Fetch Time
✓ 110	22:55:32	DELETE FROM Members WHERE MemberID...	0 row(s) affected	0.00043 sec
✓ 111	23:06:19	USE gymdb	0 row(s) affected	0.021 sec

Administration

Schemas

Query 1

gym_all_queries*

gym_database*

01_create_member.png*

SECTION_4_Full_Journey_Sample_Data*

Server Status

Client Connections

Users and Privileges

Status and System Variables

Data Export

Data Import/Restore

NCE

Object Info

Session

Object selected

Limit to 1000 rows

SQL Editor

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

USE gymdb;

SELECT

m.FirstName,

m.LastName,

c.Name AS ClassName,

CONCAT(cs.Date, ' ', cs.StartTime) AS ScheduledDateTime,

CASE

WHEN b.Attendance = 1 THEN 'Attended'

ELSE 'Missed'

END AS AttendanceStatus

FROM Bookings b

JOIN Members m ON b.MemberID = m.MemberID

JOIN ClassSchedule cs ON b.ScheduleID = cs.ScheduleID

JOIN Classes c ON cs.ClassID = c.ClassID;

100%

1:16

Result Grid

Filter Rows: Search

Export: CSV

FirstName	LastName	ClassName	ScheduledDateTime	AttendanceStat...
Maria	Silva	Spinning	2025-06-10 08:00:00	Attended
João	Costa	Yoga	2025-06-12 09:00:00	Missed
Ana	Souza	HIIT	2025-06-14 18:00:00	Missed
Carlos	Menezes	Spinning	2025-06-16 08:00:00	Attended
Laura	Ferreira	Yoga	2025-06-18 09:00:00	Missed

Result 12

Read Only